

セルフホスティングの機能

評価器をLispで表す

情報システム学科 3年・選択・2単位

今日の到達目標

- 評価器が必要とする操作を列挙できる
- 対象言語の環境とホスト環境を区別できる
- Lisp製評価器の最小例を動かして拡張できる

前回との接続

ファイル、リスト、関数を使えるようになった。

今度は評価器自身をLispのプログラムとして書く。

足りない操作を、具体的な式から見つける。

対象言語とホスト

ホスト言語は評価器を動かす言語。

対象言語は評価器が処理する言語。

両方がLispでも、それぞれ別の環境と評価段階を持つ。

当日実装と、後続の設計

区分	この回で扱う範囲
当日実装	数値、二項四則、quote、ifの最小meta
契約を設計	名前検索・定義、対象の関数と環境
後続へ接続	第13回で説明、第14回で対象関数とREPL

ホストにlambdaがあっても、対象のlambdaが実装済みとは限らない。

既存の受入例を通し、自作版の未対応機能を明記する。

機能と用途

ホスト側の機能

評価器での用途

型の判定

数値・記号・リストの分類

リスト操作

式の分解

再帰関数

式と環境の探索

入出力

結果と診断の表示

当日の型判定

対象の入力	最小metaの処理
数値	そのまま返す
記号	未対応の名前として診断
リスト	quote・if・算術の先頭を調べる

第12回参照では、真偽値と文字列も自己評価される。

対象の名前を探す環境は、契約を決めて第14回へつなぐ。

評価器の入力

入力には文字列と、読取済みのS式の2種類がある。

Lisp製評価器の中心にはS式を渡す。

文字列をS式にする入口は別に用意する。

数値の受入例

```
(define number-eval  
  (lambda (expr)  
    (if (number? expr) expr  
        (error "expected number"))))  
(number-eval 7)
```

説明用の最小例。未定義記号を黙って返さない。

quoteの受入例

対象式 `(quote (+ 1 2))` の結果はリスト `(+ 1 2)`。

対象式をホストに渡すときにも引用が必要になる。

どの層が引用を処理するかを図に書く。

二重quoteを渡すホストの呼出し

```
(m-eval (quote (quote (+ 1 2))) (m-global) 0)
```

evaluator.lispをロードしたホストで呼ぶ例。

ホストは外側のquoteを処理し、対象式をデータとして渡す。

m-globalはこの回では `()`。envはまだ名前検索に使わない。

データは各層でどう変わるか

処理する層	入力・データ	得る値
ホストのquote	<code>(quote (quote (+ 1 2)))</code>	<code>(quote (+ 1 2))</code>
対象のm-eval	<code>(quote (+ 1 2))</code>	<code>(+ 1 2)</code>
結果の表示	リスト値 <code>(+ 1 2)</code>	<code>(+ 1 2)</code> と表示

最後の値をもう一度評価しない。結果は3ではない。

3を得るには、そのリストを別の評価呼出しに渡す。

対象のquoteを分解する途中値

対象評価器へ届いたexprは `(quote (+ 1 2))`。

操作	取り出すデータ
<code>(car expr)</code>	記号quote
<code>(cdr expr)</code>	<code>((+ 1 2))</code> という引数リスト
<code>(car (cdr expr))</code>	<code>(+ 1 2)</code> という値

quote分岐は引数1個を確認し、その値をそのまま返す。

算術分岐へ再帰しないので、内部の+はまだ適用しない。

ifの受入例

対象式 `(if #f (/ 1 0) 7)` は `7` を返す。

ホストで条件を評価してしまうと、対象環境の名前を扱えない。

条件も対象評価器へ渡す。

算術は引数の内側から評価する

対象式は `(+ 1 (* 2 3))`。

途中の処理	得る値・データ
先頭+を確認	許可したホストprimitiveを取得
引数1と内側の <code>(* 2 3)</code> を評価	1と6
外側の引数値をそろえる	リスト <code>(1 6)</code>
primitiveへ値を適用	7

引数の評価順序はLisp側が決め、算術だけをホストへ委譲する。

第12回で委譲する組み込み

対象の算術名は `+` `-` `*` `/` だけを認める。

先頭の記号を確認し、primitiveと引数値をホストのapplyへ渡す。

`quote` と `if` の評価規則は、Lisp側の分岐で実装する。

未知の演算子を、ホスト環境へ黙って渡さない。

完成参照の実演

```
cd examples/12-meta-evaluator  
python3 lisp.py --meta --eval "(+ 1 (* 2 3))"
```

`evaluator.lisp` の `m-eval` が計算し、結果は `7`。

第12回の対象機能は数値・四則・引用・分岐が中心。

当日の実装チェックポイント

順序	確かめる対象の例
1 数値	7 をそのまま返す
2 quote	(quote (+ 1 2)) がリストを返す
3 二項四則	入れ子の (+ 1 (* 2 3)) が7
4 if	非選択枝の0除算を評価せず7

名前と対象関数は、ここへ同時に追加せず契約設計へ進む。

正常例だけでなく、未対応名・演算子・型の違いを調べる。

環境をデータにする：後続の設計

```
(quote ((x 3) (y 4)))
```

名前と値の組を並べた、bindingsの説明例。

この表現だけでは、親環境や更新の共有方法はまだ決まらない。

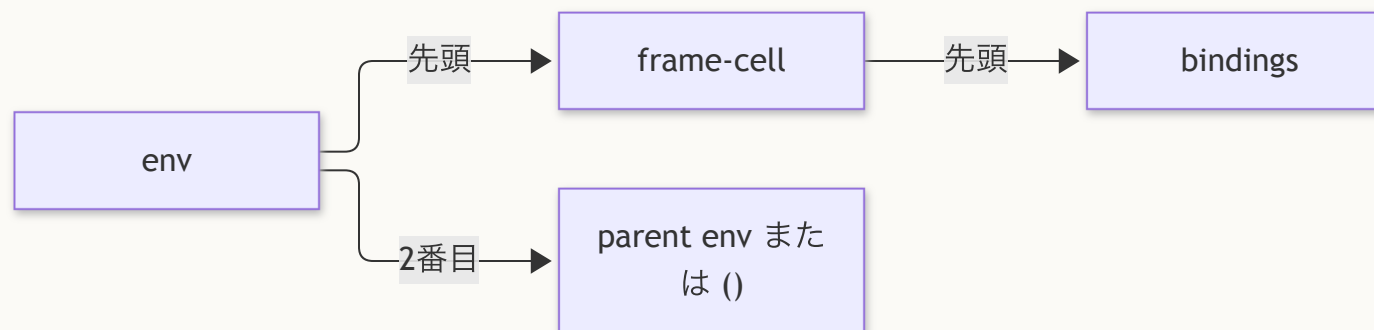
第12回のm-globalは空。以下は第14回へ向けた契約設計。

第14回参照の環境へつなぐ

段階	データの役割
bindings	<code>((x 3) (y 4))</code>
frame-cell	bindingsを包む、共有できる1要素リスト
env	<code>(frame-cell parent)</code> 、親がなければ <code>()</code>

後続の環境表現を読むための表。第12回のenvはまだ空。

束縛表を参照する道筋



後続のm-bindingsは、envの先頭の先頭を読む。

```
(m-bindings env)
(car (car env))
```

どちらも同じbindingsを得る。parentは別に保持する。

環境の操作を契約にする

操作の契約

第14回参照の対応

名前とenvから値を探す

m-lookup、親も探索

名前・値・envを定義する

m-define、フレームを更新

呼出し用の子環境を作る

m-new-envとm-bind

内部表現を操作の中に隠し、評価器の分岐から呼ぶ。

第12回は契約と反例を設計する段階。対象関数の完成は後続へ。

ホストの名前と対象の名前

ホストのexprやenvは、Lisp製評価器の実装変数。

対象の (define x 3) は、別の対象環境へ束縛を作る設計。

対象のxを、ホストの辞書からそのまま探さない。

同じ名前でも、どちらの環境を検索するかを図に書く。

通常の関数適用：後続の設計

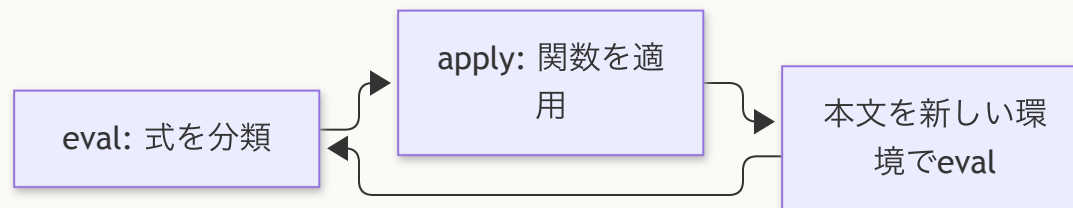
対象の先頭式を評価して、対象の関数値を得る。

対象の引数式を評価し、得た値を適用処理へ渡す。

対象関数なら、新しい環境でその本文を評価する。

第12回の算術分岐から、第14回の対象関数へ広げる契約。

evalとapply：後続の設計



式の規則と、関数の規則を分ける。

対象のクロージャ：第14回へ

対象関数にも、仮引数、本文、定義時環境が必要。

ホストの `lambda` と対象の `lambda` を区別して表す。

対象関数を呼ぶとき、その環境を拡張する。

保存と共有：第14回へ

再帰関数は、自身の定義を後から環境で見つける必要がある。

環境を単なるコピーにするか、共有するかで振る舞いが変わる。

`fact` を受入例にして判断する。

開発順序は、回ごとに分ける

回・段階	開発と説明の焦点
第12回の当日実装	数値・quote・二項四則・if
第12回の契約設計	名前・環境・対象関数の入力と失敗
第13回の理解確認	自作最小metaと完成参照を区別して議論
第14回の統合	対象の名前・関数、Lisp製REPL

各段階の達成と未対応を記録し、完成参照の観察と混同しない。

演習1 機能契約

必要な操作を1個選び、入力・出力・失敗条件を書く。

その契約を使う対象式を示す。

便利だから追加する、という説明だけでは足りない。

演習2 最小評価器

数値と二項加算が通る入口を作る。

次に未定義名と未知の演算子を試す。

どの機能をまだ扱えないか明記する。

演習3 段階拡張

対象の `if` を追加し、非選択枝のゼロ除算を避ける。

名前と関数の契約を設計し、第13回の理解確認を経て第14回で統合する。

ホスト環境との混同を検出する。

確認問い

なぜホストの評価関数へ対象式を渡すだけでは不十分か。

組込みを委譲するとき、Lispで実装した範囲をどう説明するか。

解答 Lisp製評価器の担当

```
(if #f (/ 1 0) (+ 1 (* 2 3)))
```

第12回の最小metaでも結果は 7。非選択枝のゼロ除算は起こらない。

Lisp側が `quote`、`if` の評価規則と引数の再帰評価を決める。算術primitiveへは、評価済みの数値を渡す。

対象式全体をホスト評価器へ渡すだけの案では、これらの意味を自分のLisp評価器で実装したことにならない。

実装の境界

現段階はPython製Lispホスト上で動くLisp製評価器。

読取・数値表現・OS入出力などのホストへの依存が残る。

ネイティブ起動や独立ブートストラップの達成とは区別して報告する。

まとめと次回

数値・四則・quote・ifを、Lisp製の最小評価器で扱った。

名前・環境・対象関数は契約を設計し、第14回へつなぐ。

次回は自作版と完成参照を区別し、予測と反例で理解を確認する。

参考

- 本教材: `examples/12-meta-evaluator/`
- SICP メタ循環評価器
- The Scheme Programming Language

参照資料と教材方言の機能差は仕様書で確認する。